

INTERVIEW PREPARATION GUIDE



MONGODB

INTERVIEW PREP

Questions & Answers

Every core MongoDB interview topic — from documents and CRUD to indexing, aggregation, sharding, and transactions — with complete, plain-English answers you can actually study from.

10 Categories

57 Questions

Full Answers



@MohitDecodes

Table of Contents

Organized by topic, from fundamentals through aggregation, replication, sharding, and security — study in order or jump to what you need.

01

MongoDB Fundamentals

Document model, BSON, SQL vs. NoSQL

6 Q

02

Documents & Collections

_id, ObjectId, document size limit, schema flexibility

6 Q

03

CRUD Operations

Insert, find, update, delete, upsert, bulkWrite

7 Q

04

Querying & Operators

Query operators, logical operators, arrays, updates

5 Q

05

Indexing

Compound indexes, multikey indexes, explain()

5 Q

06

Aggregation Framework

\$match, \$group, \$project, \$lookup

6 Q

07

Data Modeling & Schema Design

Embedding vs. referencing, validation, denormalization

6 Q

08

Replication & High Availability

Replica sets, failover, oplog, write concern

5 Q

09

Sharding & Scalability

Shard keys, mongos, config servers, chunk migration

5 Q

10

Transactions, Performance & Security

ACID transactions, profiling, RBAC

6 Q

Q1 What is MongoDB and how does it differ from a relational database?

MongoDB is a document-oriented NoSQL database that stores data as flexible, JSON-like documents (BSON) instead of rows in fixed tables. Unlike a relational database, it doesn't require a predefined schema, and related data is often embedded within a single document rather than split across tables and joined.

Q2 What is BSON and why does MongoDB use it instead of JSON?

BSON (Binary JSON) is a binary-encoded serialization format MongoDB uses internally to store documents. It extends JSON with additional types, like dates and binary data, and is faster to parse and more compact than storing plain text JSON.

Q3 What are the main components of MongoDB's structure?

A MongoDB deployment contains databases, each database contains collections, and each collection holds documents — roughly analogous to a database containing tables (collections) made up of rows (documents) in the relational world, though far more flexible.

Q4 What is the difference between SQL and NoSQL databases?

SQL databases are relational, using fixed schemas and tables connected through foreign keys, queried with SQL. NoSQL databases, including MongoDB, favor flexible or schema-less data models — document, key-value, column-family, or graph — often optimized for horizontal scaling and rapidly changing data shapes.

Q5 What are the advantages of using MongoDB?

A flexible schema that adapts as requirements change, a natural mapping to objects in application code, horizontal scalability through sharding, high performance for read/write-heavy workloads, and a rich query language including aggregation pipelines.

Q6 What is mongosh?

mongosh is MongoDB's modern command-line shell, used to connect to a MongoDB instance, run queries and administrative commands, and interact with the database directly, replacing the legacy mongo shell.

Q1 What is a document in MongoDB?

A document is the basic unit of data in MongoDB — a set of field-value pairs, similar to a JSON object, stored in BSON format. Each document is stored inside a collection and can have its own distinct set of fields.

Q2 What is a collection in MongoDB?

A collection is a grouping of MongoDB documents, roughly equivalent to a table in a relational database — but collections don't enforce a fixed schema, so documents within the same collection can have different fields.

Q3 What is the `_id` field?

Every document has a unique `_id` field acting as its primary key. If you don't provide one, MongoDB automatically generates an ObjectId for it when the document is inserted.

Q4 What is an ObjectId and what information does it encode?

A 12-byte identifier MongoDB generates by default for `_id`. It encodes a timestamp, a random value, and an incrementing counter, making it unique across a distributed system without needing a centralized counter.

Q5 What is the maximum size of a MongoDB document?

16 megabytes per document — a limit designed to keep documents efficient to send over the network and to prevent an excessive amount of RAM being used for a single document.

Q6 Does MongoDB enforce a schema on a collection?

Not by default — MongoDB is schema-flexible, meaning documents in the same collection can have different structures. However, you can enforce structure and validation rules using JSON Schema validation on a collection if consistency is required.

Q1 How do you insert a document into a MongoDB collection?

Use `insertOne()` for a single document or `insertMany()` for multiple documents — e.g., `db.users.insertOne({ name: "Alice", age: 30 })`.

Q2 How do you find documents in MongoDB?

Use `find()` with a query filter to return matching documents, or `findOne()` to return just the first match — e.g., `db.users.find({ age: { $gt: 25 } })`.

Q3 How do you update documents in MongoDB?

`updateOne()` or `updateMany()` modify existing documents that match a filter, typically using update operators like `$set` — e.g., `db.users.updateOne({ name: "Alice" }, { $set: { age: 31 } })`.

Q4 What is the difference between `updateOne()`, `updateMany()`, and `replaceOne()`?

`updateOne()` and `updateMany()` modify specific fields of one or many matching documents using operators like `$set`. `replaceOne()` replaces the entire matched document's content with a new document, except for its `_id`.

Q5 How do you delete documents in MongoDB?

`deleteOne()` removes the first document matching a filter, and `deleteMany()` removes all matching documents — e.g., `db.users.deleteMany({ status: "inactive" })`.

Q6 What is upsert and when would you use it?

An upsert (update + insert) is an option on update operations — if no document matches the filter, a new document is inserted instead of the update doing nothing. Useful for "create or update" logic in a single call.

Q7 What does the `bulkWrite()` method do?

It lets you batch multiple insert, update, and delete operations into a single request to the server, reducing round trips and improving performance compared to issuing each operation separately.

Q1 What are query operators in MongoDB, and can you give examples?

Special keys prefixed with \$ used inside query filters to express conditions beyond simple equality — like \$gt, \$lt, \$in, \$ne, \$exists, and \$regex — e.g., { age: { \$gte: 18, \$lte: 65 } }.

Q2 What is the difference between \$and, \$or, and \$nor?

\$and requires all given conditions to be true, \$or requires at least one to be true, and \$nor requires all given conditions to be false — each takes an array of condition objects.

Q3 How do you query for a field inside a nested (embedded) document?

Use dot notation on the field path — e.g., { "address.city": "Delhi" } matches documents where the nested address document has city equal to "Delhi".

Q4 How do you query for a value inside an array field?

A simple equality check like { tags: "mongodb" } matches if the array contains that value anywhere. Operators like \$elemMatch let you require multiple conditions to match on the same array element.

Q5 What is the difference between \$set, \$unset, \$inc, and \$push in update operations?

\$set assigns a new value to a field, \$unset removes a field entirely, \$inc increments a numeric field by a given amount, and \$push appends a value to an array field.

Q1 What is an index in MongoDB and why does it matter?

An index is a data structure that stores a small, ordered portion of a collection's data, letting MongoDB find matching documents without scanning every document — dramatically speeding up queries at the cost of extra storage and slightly slower writes.

Q2 What is the default index on every MongoDB collection?

A unique index on the `_id` field, created automatically when a collection is made, which MongoDB uses to enforce uniqueness and speed up lookups by `_id`.

Q3 What is a compound index?

An index built on multiple fields at once, like `{ lastName: 1, firstName: 1 }`, useful for queries that filter or sort on that same combination of fields — the order of fields in the index matters for which queries it can support efficiently.

Q4 What is the difference between a single-field index and a multikey index?

A single-field index is built on one scalar field. A multikey index is automatically created when you index a field that holds an array — MongoDB indexes each element of the array separately so queries can match any of them.

Q5 How would you find out if a query is using an index efficiently?

Run the query with `.explain()`, e.g., `db.users.find({...}).explain("executionStats")`, to see whether MongoDB used an index scan (IXSCAN) or a full collection scan (COLLSCAN), along with how many documents were examined versus returned.

Q1 What is the aggregation pipeline in MongoDB?

A framework for processing documents through a sequence of stages — each stage transforms the documents and passes the result to the next — used for tasks like filtering, grouping, reshaping, and computing summary statistics.

Q2 What does the \$match stage do, and where should it typically appear in a pipeline?

\$match filters documents, similar to a query filter, keeping only those that match given conditions. Placing it early in the pipeline reduces the number of documents later stages have to process, which improves performance.

Q3 What does the \$group stage do?

It groups documents by a specified key, or combination of keys, and computes aggregated values per group, using accumulator operators like \$sum, \$avg, \$max, \$min, and \$push.

Q4 What is the difference between \$project and \$group?

\$project reshapes each document individually — including, excluding, or computing fields — without changing the number of documents. \$group collapses many documents into fewer documents, one per distinct group key.

Q5 What does the \$lookup stage do?

It performs a left outer join with another collection in the same database, letting you pull in related documents — commonly used to combine data that's normalized across collections instead of embedded.

Q6 What is the difference between the aggregation pipeline and simple find() queries?

find() is for straightforward retrieval and filtering of documents as they are. The aggregation pipeline can filter, reshape, group, join, and compute derived values across multiple stages — suited for analytics and more complex data transformations that find() alone can't express.

Q1 What is the difference between embedding and referencing in MongoDB schema design?

Embedding stores related data directly inside a parent document (denormalized), which is fast to read in one query but can duplicate data. Referencing stores related data in a separate collection and links to it by `_id` (normalized), avoiding duplication but requiring a separate query or `$lookup` to join.

Q2 When would you choose embedding over referencing?

When the related data is small, doesn't change often, is always accessed together with the parent, and won't grow unbounded — like an address embedded in a user document, or line items embedded in an order.

Q3 When would you choose referencing over embedding?

When the related data is large, shared across many parent documents, updated independently and frequently, or could grow without bound inside a single document — like comments on a highly popular post, or products referenced from many orders.

Q4 What is schema validation in MongoDB, and how do you apply it?

A way to enforce structure and rules on documents in a collection, defined using JSON Schema and applied via the `validator` option when creating or modifying a collection — rejecting inserts and updates that don't conform, while still allowing more flexibility than a rigid relational schema.

Q5 What is denormalization and why is it common in MongoDB?

Storing the same piece of data in multiple places to avoid extra lookups at read time. It's common in MongoDB because reads are often optimized by keeping everything needed for a common query pattern in one document, at the cost of needing to update multiple places when that data changes.

Q6 What is the 16MB document size limit's practical impact on schema design?

It discourages unbounded embedding — like embedding every comment ever made on a post directly inside that post's document — pushing you toward referencing or capping the embedded array once a collection of sub-documents could grow very large.

Q1 What is a replica set in MongoDB?

A group of mongod processes that maintain the same data set, providing redundancy and high availability. One member is the primary, accepting all writes, while the others are secondaries that replicate the primary's data.

Q2 How does failover work in a MongoDB replica set?

If the primary becomes unreachable, the remaining members hold an election and promote one of the eligible secondaries to primary automatically, allowing the replica set to keep accepting writes with minimal manual intervention.

Q3 What is the oplog?

The operations log — a special capped collection on the primary that records every write operation, which secondaries continuously read and apply to replicate the primary's data in near real time.

Q4 What is read preference in a replica set, and why would you change it?

A setting that controls which replica set members a client can read from — primary (default, most consistent), primaryPreferred, secondary, secondaryPreferred, or nearest — useful for distributing read load or serving reads from a geographically closer node, at the cost of potentially reading slightly stale data.

Q5 What is write concern, and how does it relate to durability?

Write concern specifies how many replica set members must acknowledge a write before MongoDB considers it successful — e.g., w: 1 (just the primary) versus w: "majority" (most of the set) — trading off write latency against the durability guarantee that the write survives a failover.

Q1 What is sharding in MongoDB?

A method of horizontally scaling MongoDB by distributing data across multiple servers, called shards, so no single machine needs to store or process the entire dataset — used when a dataset or workload outgrows what one replica set can efficiently handle.

Q2 What is a shard key, and why is choosing one carefully important?

The field, or fields, MongoDB uses to distribute documents across shards. A poorly chosen shard key can create "hot" shards that receive disproportionate traffic or data, undermining the whole point of sharding — a good key distributes both data and load evenly.

Q3 What are the main components of a sharded cluster?

Shards, each typically a replica set storing a portion of the data; mongos, a query router that directs operations to the right shard or shards; and config servers, which store the cluster's metadata, like which data lives on which shard.

Q4 What is the difference between a hashed shard key and a ranged shard key?

A ranged shard key distributes documents by contiguous ranges of the key's value, which supports efficient range queries but risks uneven distribution if writes cluster around one range. A hashed shard key distributes documents based on a hash of the key's value, spreading writes more evenly but making range queries across shards less efficient.

Q5 What is chunk migration in a sharded cluster?

MongoDB splits shard data into chunks based on shard key ranges. As data grows unevenly across shards, the balancer process automatically migrates chunks between shards to keep the data distribution even.

Q1 Does MongoDB support multi-document ACID transactions?

Yes — since MongoDB 4.0 for replica sets, and 4.2 for sharded clusters, MongoDB supports multi-document transactions with full ACID guarantees, letting you group multiple operations across documents or collections so they all succeed or all roll back together.

Q2 When would you actually need a multi-document transaction in MongoDB?

When an operation must atomically update multiple documents or collections together, like transferring funds between two account documents — though many use cases are better solved by modeling data so a single-document write, which is always atomic, is sufficient.

Q3 What tools does MongoDB provide for monitoring performance?

The `.explain()` method for individual queries, the database profiler for logging slow operations, `mongostat` and `mongotop` for real-time server statistics, and MongoDB Atlas's built-in performance advisor and monitoring dashboards for cloud deployments.

Q4 What is the MongoDB profiler used for?

It records detailed information about database operations — like query shape, execution time, and documents examined — letting you identify slow queries that might benefit from an index or a rewritten query.

Q5 What are common MongoDB security best practices?

Enabling authentication and role-based access control, encrypting data in transit with TLS and at rest, restricting network access through specific IP bindings or a VPC/firewall, following the principle of least privilege for user roles, and keeping MongoDB updated with security patches.

Q6 What is role-based access control (RBAC) in MongoDB?

A system where users are granted specific roles, like `read`, `readWrite`, or `dbAdmin`, scoped to particular databases or collections, rather than broad access to everything — letting you grant only the permissions each user or application actually needs.